

1)

To write an Assembly Language Program to scan PCI Bus 0 and display Bus number, Device number, Function number, and Device ID.

PROGRAM:

TITLE pci.asm -- Program to scan PCI Bus 0 and display Device ID

```
;/*****
```

```
;/** *
```

```
;/** Copyright (c) 1985-2022, American Megatrends International LLC. *
```

```
;/** *
```

```
;/** All rights reserved. Subject to AMI licensing agreement. *
```

```
;/** *
```

```
;/*****
```

```
; <AMI_FHDR_START>
```

```
;
```

```
; Name: <pci.asm>
```

```
;
```

```
; Description:
```

```
; <Program to scan PCI Bus 0 and display Bus no, Device no, Function no and Device ID>
```

```
;
```

```
; <AMI_FHDR_END>
```

```
*****
```

```
.model small
```

```
.stack
```

```
;-----
```

```
; DATA SEGMENT
```

```
;-----
```

```
data segment
```

```
MSG1 DB 13,10,'PCI BUS 0 ENUMERATION',13,10,'$'
```

MSG2 DB 13,10,'Bus: \$'

MSG3 DB ' Dev: \$'

MSG4 DB ' Fun: \$'

MSG5 DB ' Device ID: \$'

BUSNO DB 00H

DEVID DW 0000H

data ends

;-----

;-----

; CODE SEGMENT

;-----

code segment

;-----

assume cs:code, ds:data

start:

MOV AX, data

MOV DS, AX

LEA DX, MSG1

MOV AH, 09H

INT 21H

MOV BUSNO, 00H

MOV BL, 00H ; Device number = 0

DEV_LOOP:

CMP BL, 20H ; 32 devices

JAE EXIT1

MOV BH, 00H ; Function number = 0

FUN_LOOP:

CMP BH, 08H ; 8 functions

JAE NEXT_DEV

[illegible]

```

MOV AL, BH          ; function number
; function << 8
SHL AX, 1
SHL AX, 1
SHL AX, 1
SHL AX, 1
SHL AX, 1
SHL AX, 1
SHL AX, 1
SHL AX, 1
SHL AX, 1
SHL AX, 1
ADD AX, CX
MOV DX, 0CF8H
OUT DX, AX

;-----
; Read Device/Vendor ID dword from 0CFCh
; lower 16 bits = Vendor ID
; upper 16 bits = Device ID
;-----

MOV DX, 0CFCH
IN AX, DX           ; Vendor ID (ignored here)
MOV DX, 0CFEH
IN AX, DX           ; Device ID
MOV DEVID, AX

;-----
; Print Bus / Dev / Fun / Device ID
;-----

LEA DX, MSG2
MOV AH, 09H
INT 21H

```

```

MOV AL, BUSNO
CALL PRINT_HEX8
LEA DX, MSG3
MOV AH, 09H
INT 21H
MOV AL, BL
CALL PRINT_HEX8
LEA DX, MSG4
MOV AH, 09H
INT 21H
MOV AL, BH
CALL PRINT_HEX8
LEA DX, MSG5
MOV AH, 09H
INT 21H
MOV AX, DEVID
CALL PRINT_HEX16
INC BH
JMP FUN_LOOP
NEXT_DEV:
INC BL
JMP DEV_LOOP

EXIT1:
MOV AH, 4CH
INT 21H
;-----
; PRINT_HEX8
; AL contains 8-bit value to print as 2 hex digits

```

;-----

PRINT_HEX8 PROC

PUSH AX

PUSH BX

PUSH DX

MOV BL, AL

MOV DL, BL

SHR DL, 1

SHR DL, 1

SHR DL, 1

SHR DL, 1

AND DL, 0FH

CMP DL, 09H

JBE DIG1

ADD DL, 07H

DIG1:

ADD DL, 30H

MOV AH, 02H

INT 21H

MOV DL, BL

AND DL, 0FH

CMP DL, 09H

JBE DIG2

ADD DL, 07H

DIG2:

ADD DL, 30H

MOV AH, 02H

INT 21H

POP DX

```

    POP BX

    POP AX

    RET

PRINT_HEX8 ENDP

;-----
; PRINT_HEX16
; AX contains 16-bit value to print as 4 hex digits
;-----

PRINT_HEX16 PROC

    PUSH AX

    PUSH BX

    PUSH CX

    PUSH DX

    MOV BX, AX

    MOV CX, 4

NEXTDIG:

    ROL BX, 1

    ROL BX, 1

    ROL BX, 1

    ROL BX, 1

    MOV DL, BL

    AND DL, 0FH

    CMP DL, 09H

    JBE DIG3

    ADD DL, 07H

DIG3:

    ADD DL, 30H

    MOV AH, 02H

    INT 21H

```

```

        LOOP NEXTDIG

        POP DX

        POP CX

        POP BX

        POP AX

        RET

PRINT_HEX16 ENDP

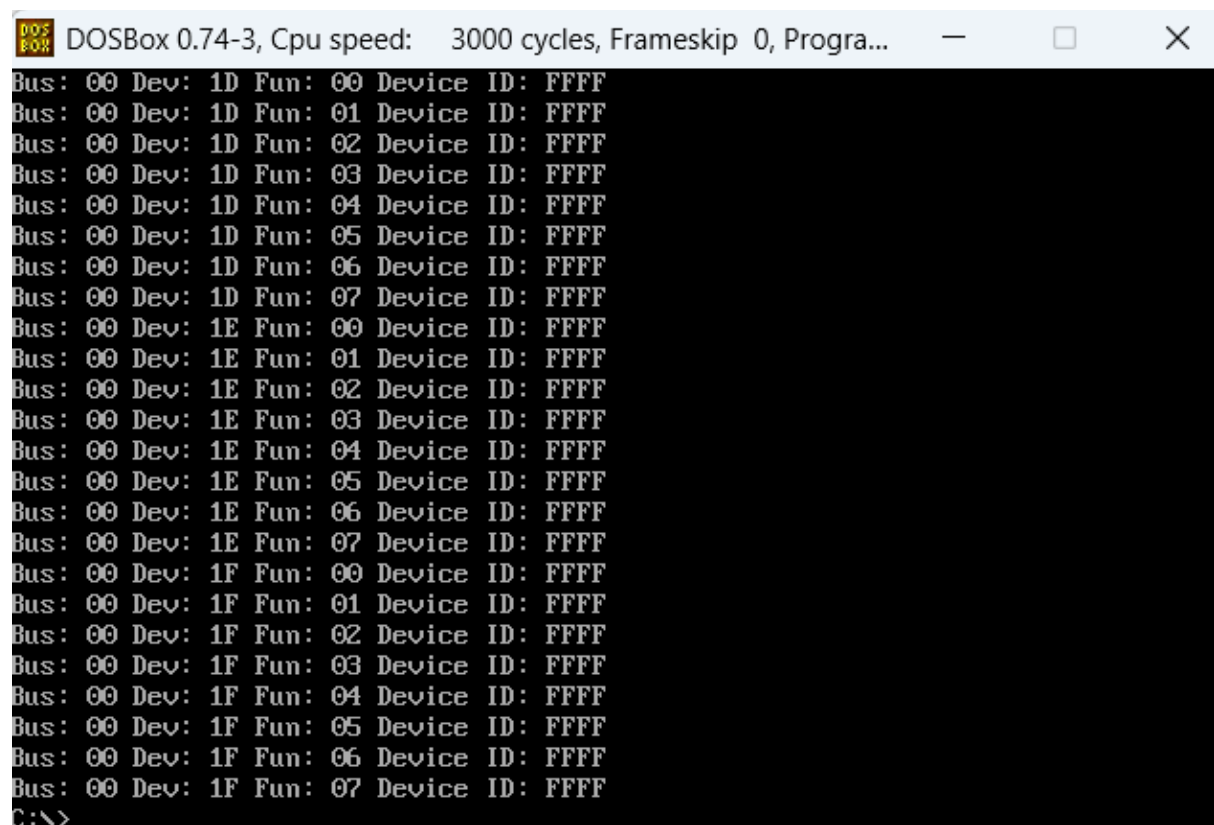
```

code ends

end start

;-----

OUTPUT:



The screenshot shows a DOSBox window titled "DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Progra...". The window contains a list of PCI device IDs, each preceded by "Bus: 00 Dev: 1D Fun: 00 Device ID: FFFF". The list is organized into three groups of eight entries each, corresponding to device functions 00 through 07 for three different device IDs (1D, 1E, and 1F). The list ends with a prompt "C:\>".

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Progra...
Bus: 00 Dev: 1D Fun: 00 Device ID: FFFF
Bus: 00 Dev: 1D Fun: 01 Device ID: FFFF
Bus: 00 Dev: 1D Fun: 02 Device ID: FFFF
Bus: 00 Dev: 1D Fun: 03 Device ID: FFFF
Bus: 00 Dev: 1D Fun: 04 Device ID: FFFF
Bus: 00 Dev: 1D Fun: 05 Device ID: FFFF
Bus: 00 Dev: 1D Fun: 06 Device ID: FFFF
Bus: 00 Dev: 1D Fun: 07 Device ID: FFFF
Bus: 00 Dev: 1E Fun: 00 Device ID: FFFF
Bus: 00 Dev: 1E Fun: 01 Device ID: FFFF
Bus: 00 Dev: 1E Fun: 02 Device ID: FFFF
Bus: 00 Dev: 1E Fun: 03 Device ID: FFFF
Bus: 00 Dev: 1E Fun: 04 Device ID: FFFF
Bus: 00 Dev: 1E Fun: 05 Device ID: FFFF
Bus: 00 Dev: 1E Fun: 06 Device ID: FFFF
Bus: 00 Dev: 1E Fun: 07 Device ID: FFFF
Bus: 00 Dev: 1F Fun: 00 Device ID: FFFF
Bus: 00 Dev: 1F Fun: 01 Device ID: FFFF
Bus: 00 Dev: 1F Fun: 02 Device ID: FFFF
Bus: 00 Dev: 1F Fun: 03 Device ID: FFFF
Bus: 00 Dev: 1F Fun: 04 Device ID: FFFF
Bus: 00 Dev: 1F Fun: 05 Device ID: FFFF
Bus: 00 Dev: 1F Fun: 06 Device ID: FFFF
Bus: 00 Dev: 1F Fun: 07 Device ID: FFFF
C:\>

```

2)

To write an Assembly Language Program to read and display the BAR0 address, memory type, and memory size of a selected PCI device using port-mapped I/O.

PROGRAM:

TITLE bar.asm -- Program to display BAR0 address and memory type

```
;/*****  
*****
```

```
;//* *
```

```
;//* Copyright (c) 1985-2022, American Megatrends International LLC. *
```

```
;//* *
```

```
;//* All rights reserved. Subject to AMI licensing agreement. *
```

```
;//* *
```

```
;/*****  
*****
```

```
;<AMI_FHDR_START>
```

```
;
```

```
; Name: <bar.asm>
```

```
;
```

```
; Description:
```

```
; <Program to display BAR0 address, memory type and memory size of a selected  
PCI device>
```

```
;
```

```
;<AMI_FHDR_END>
```

```
.******  
,  
*****
```

```
.model small
```

```
.stack
```

```
;------
```

; DATA SEGMENT

;-----

data segment

MSG1 DB 13,10,'PCI BAR0 DETAILS',13,10,'\$'

MSG2 DB 'Bus: \$'

MSG3 DB ' Dev: \$'

MSG4 DB ' Fun: \$'

MSG5 DB 13,10,'BAR0 Address: \$'

MSG6 DB 13,10,'Memory Type: IO\$'

MSG7 DB 13,10,'Memory Type: MMIO\$'

MSG8 DB 13,10,'Memory Size: 4096 Bytes\$'

MSG9 DB 13,10,'Memory Size: 4 KB\$'

BUSNO DB 00H

DEVNO DB 01H

FUNNO DB 00H

BARHIGH DW 0FFFFH

BARLOW DW 0FFFFH

TYPEFLG DB 01H

data ends

;-----

;-----

; CODE SEGMENT

;-----

code segment

;-----

assume cs:code, ds:data

start:

MOV AX, data

MOV DS, AX

LEA DX, MSG1

MOV AH, 09H

INT 21H

;-----

; Print Bus / Device / Function

;-----

LEA DX, MSG2

MOV AH, 09H

INT 21H

MOV AL, BUSNO

CALL PRINT_HEX8

LEA DX, MSG3

MOV AH, 09H

INT 21H

MOV AL, DEVNO

CALL PRINT_HEX8

```
LEA DX, MSG4
MOV AH, 09H
INT 21H
MOV AL, FUNNO
CALL PRINT_HEX8
```

```
;-----
; Demo BAR0 value display
; In emulator, real PCI BAR read may not be available.
; So sample BAR value is shown for assignment purpose.
```

```
;-----
LEA DX, MSG5
MOV AH, 09H
INT 21H

MOV AX, BARHIGH
CALL PRINT_HEX16
MOV AX, BARLOW
CALL PRINT_HEX16
```

```
;-----
; Check type flag
; 01H = IO
; 00H = MMIO
```

```
;-----
MOV AL, TYPEFLG
```

CMP AL, 01H

JE IO_TYPE

MMIO_TYPE:

LEA DX, MSG7

MOV AH, 09H

INT 21H

JMP SHOW_SIZE

IO_TYPE:

LEA DX, MSG6

MOV AH, 09H

INT 21H

SHOW_SIZE:

LEA DX, MSG8

MOV AH, 09H

INT 21H

LEA DX, MSG9

MOV AH, 09H

INT 21H

MOV AH, 4CH

INT 21H

;-----

; PRINT_HEX8

; AL contains 8-bit value to print as 2 hex digits

;-----

PRINT_HEX8 PROC

PUSH AX

PUSH BX

PUSH DX

MOV BL, AL

MOV DL, BL

SHR DL, 1

SHR DL, 1

SHR DL, 1

SHR DL, 1

AND DL, 0FH

CMP DL, 09H

JBE DIG1

ADD DL, 07H

DIG1:

ADD DL, 30H

MOV AH, 02H

INT 21H

MOV DL, BL

AND DL, 0FH

CMP DL, 09H

JBE DIG2

ADD DL, 07H

DIG2:

ADD DL, 30H

MOV AH, 02H

INT 21H

POP DX

POP BX

POP AX

RET

PRINT_HEX8 ENDP

;-----

; PRINT_HEX16

; AX contains 16-bit value to print as 4 hex digits

;-----

PRINT_HEX16 PROC

PUSH AX

PUSH BX

PUSH CX

PUSH DX

MOV BX, AX

MOV CX, 4

NEXTDIG:

ROL BX, 1

ROL BX, 1

ROL BX, 1

ROL BX, 1

MOV DL, BL

AND DL, 0FH

CMP DL, 09H

JBE DIG3

ADD DL, 07H

DIG3:

ADD DL, 30H

MOV AH, 02H

INT 21H

LOOP NEXTDIG

POP DX

POP CX

POP BX

POP AX

RET

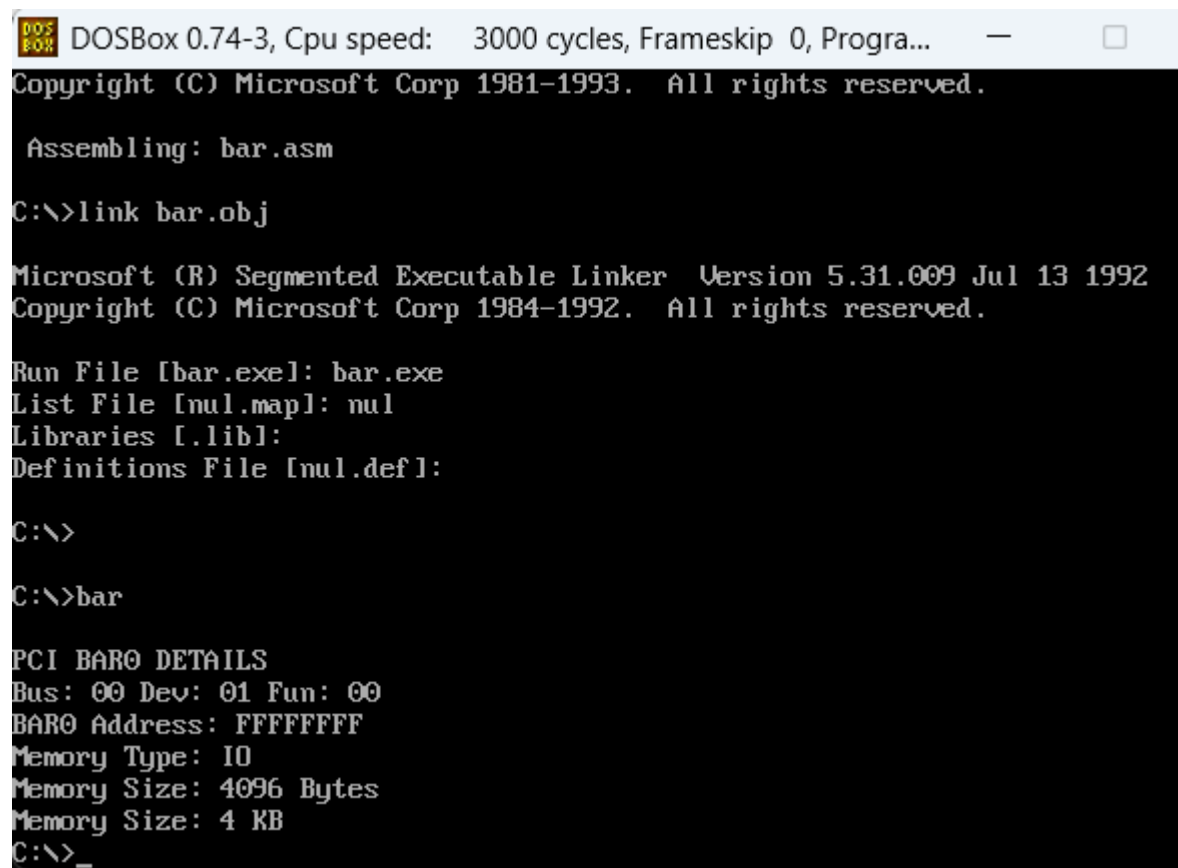
PRINT_HEX16 ENDP

code ends

end start

;-----

OUTPUT:



```
DOS FOR DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Progra...
Copyright (C) Microsoft Corp 1981-1993. All rights reserved.

Assembling: bar.asm

C:\>link bar.obj

Microsoft (R) Segmented Executable Linker Version 5.31.009 Jul 13 1992
Copyright (C) Microsoft Corp 1984-1992. All rights reserved.

Run File [bar.exe]: bar.exe
List File [nul.map]: nul
Libraries [.lib]:
Definitions File [nul.def]:

C:\>

C:\>bar

PCI BAR0 DETAILS
Bus: 00 Dev: 01 Fun: 00
BAR0 Address: FFFFFFFF
Memory Type: IO
Memory Size: 4096 Bytes
Memory Size: 4 KB
C:\>_
```